

SDM18和Jetson通信

SDM18和Jetson通信

SDM18简介

1. 实验准备
2. 实验接线
3. 实验步骤
4. 实验结果
5. SDM18的一些重要参数与默认值讲解

SDM18简介

- YDLIDAR SDM18 激光雷达是的一款高性能单点激光雷达（以下简称：SDM18）。本产品基于飞行时间测距原理，并配以相关光学、电学、算法设计，实现高精度激光距离测量，并输出高帧率的点云数据。
- 产品特性
 - 测距频率高，内部高采样率结合滤波算法，数据具有较高稳定性
 - 探测距离远，可达 18 米远
 - 重量轻，约 1.35g
 - 激光功率满足 FDA Class I 安全标准

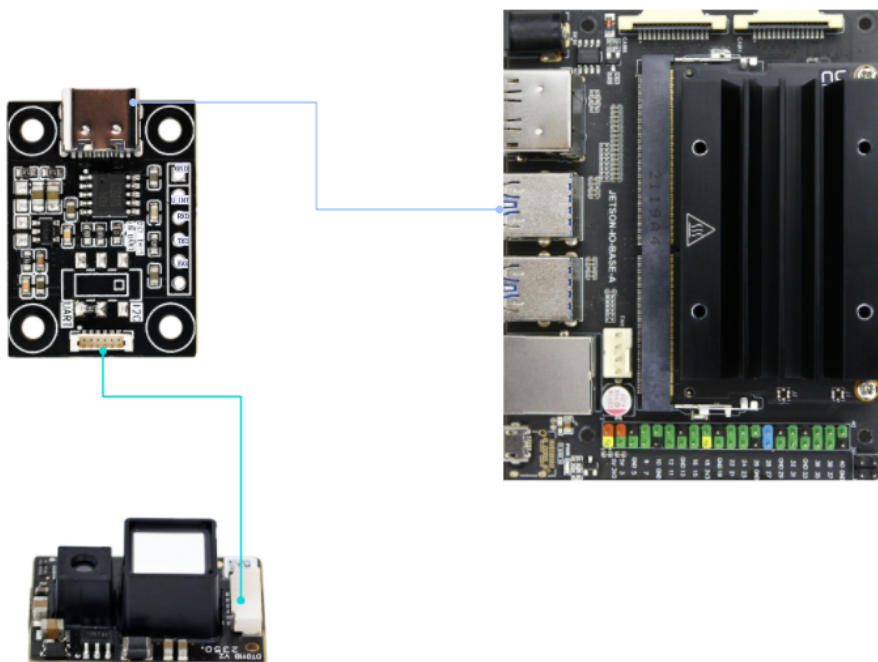
1. 实验准备

- SDM18激光测距模块
- Jetson系列 本教程实验的是Jetson Orin Nano

2. 实验接线

注意：串口转接板可以自己焊接排针，默认出厂是不带排针的，我们测试无法使用引脚通信需要使用USB端口，这里直接通过Type-C连接主板和SDM18(UART波特率为921600)

接线图如图所示



3.实验步骤

1. 把例程提供的sdm18_test.py程序放到jetson上
2. 接上USB后输入lsusb, 识别到如下设备说明连接成功

```
jetson@yahboom:~$ lsusb
Bus 002 Device 002: ID 2109:0822 VIA Labs, Inc. USB3.1 Hub
Bus 002 Device 001: ID 1d6b:0003 Linux Foundation 3.0 root hub
Bus 001 Device 003: ID 0bda:c822 Realtek Semiconductor Corp. Bluetooth Radio
Bus 001 Device 006: ID 2109:8822 VIA Labs, Inc. USB Billboard Device
Bus 001 Device 012: ID 1a86:7522 QinHeng Electronics USB Serial
Bus 001 Device 002: ID 2109:2822 VIA Labs, Inc. USB2.0 Hub
Bus 001 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
```

3. ls /dev/ttyUSB*看是否能识别到USB设备, 如果可以识别跳过第四步, 直接进行第五步, 不行的话需要按照第四步方法操作
4. 输入lsusb -t

```
jetson@yahboom:~/test$ lsusb
Bus 002 Device 002: ID 2109:0822 VIA Labs, Inc. USB3.1 Hub
Bus 002 Device 001: ID 1d6b:0003 Linux Foundation 3.0 root hub
Bus 001 Device 003: ID 0bda:c822 Realtek Semiconductor Corp. Bluetooth Radio
Bus 001 Device 004: ID 2109:8822 VIA Labs, Inc. USB Billboard Device
Bus 001 Device 005: ID 1a86:7522 QinHeng Electronics USB Serial
Bus 001 Device 002: ID 2109:2822 VIA Labs, Inc. USB2.0 Hub
Bus 001 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
jetson@yahboom:~/test$ lsusb -t
/: Bus 02.Port 1: Dev 1, Class=root_hub, Driver=tegra-xusb/4p, 10000M
|__ Port 1: Dev 2, If 0, Class=Hub, Driver=hub/4p, 10000M
/: Bus 01.Port 1: Dev 1, Class=root_hub, Driver=tegra-xusb/4p, 480M
|__ Port 2: Dev 2, If 0, Class=Hub, Driver=hub/5p, 480M
|__ Port 5: Dev 4, If 0, Class=, Driver=, 480M
|__ Port 3: Dev 5, If 0, Class=Vendor Specific Class, Driver=, 12M
|__ Port 3: Dev 3, If 0, Class=Wireless, Driver=btusb, 12M
|__ Port 3: Dev 3, If 1, Class=Wireless, Driver=btusb, 12M
jetson@yahboom:~/test$
```

git clone <https://github.com/juliagoda/CH341SER.git>

cd CH341SER

make -j6

sudo make install

sudo modprobe ch34x

然后输入modinfo ch34x

成功的话会有以下信息

```
jetson@yahboom:~/test$ modinfo ch34x
filename:       /lib/modules/5.15.148-tegra/kernel/drivers/usb/serial/ch34x.ko
author:        <tech@wch.cn>
description:    WCH CH34x USB to serial adaptor driver
license:       GPL
alias:         usb:v1A86p5523d*dc*dsc*dp*ic*isc*ip*in*
alias:         usb:v1A86p7523d*dc*dsc*dp*ic*isc*ip*in*
depends:        usbserial
name:          ch34x
vermagic:      5.15.148-tegra SMP preempt mod_unload modversions aarch64
```

可以先执行以下命令 看是否会出现'ttyUSB*'这样的设备如果成功的话继续按照下面的步骤执行, 如果还是不成功请通过引脚连接usb转ttl (GND-->GND RXD-->TX TXD-->RX 3V3 -->3.3V)

```
sudo sh -c 'echo "1a86 7522" > /sys/bus/usb-serial/drivers/ch34x/new_id'
```

```
jetson@yahboom:~$ ls /dev/ttyU*
/dev/ttyUSB0
```

#1. 创建绑定脚本

```
sudo tee /usr/local/bin/ch34x_bind.sh <<EOF
```

```
#!/bin/bash
sudo modprobe ch34x
echo "1a86 7522" > /sys/bus/usb-serial/drivers/ch34x/new_id
EOF

# 赋予执行权限
sudo chmod +x /usr/local/bin/ch34x_bind.sh

#2. 通过 cron实现开机自启
# 编辑 cron 任务
sudo crontab -e
在文件末尾添加:

@reboot /usr/local/bin/ch34x_bind.sh
```

5. 需要根据实际的设备号修改程序如/dev/ttyUSB0
6. 实验需要用到Python环境以及pyserial库需要自行准备

```
pip3 install pyserial
```

7. 需要提前把sdm18_test.py程序上传到jetson上, 然后终端输入以下命令

```
python3 sdm18_test.py
```

4.实验结果

把线接好后, 正常通信的情况下, 串口助手是能打印出得到的测距信息, 结果如图所示

```
Distance: 306 mm, Strength: 20396
Distance: 207 mm, Strength: 12208
Distance: 188 mm, Strength: 15034
Distance: 135 mm, Strength: 20489
Distance: 102 mm, Strength: 17530
Distance: 122 mm, Strength: 18883
Distance: 109 mm, Strength: 16209
Distance: 105 mm, Strength: 15849
Distance: 105 mm, Strength: 16771
Distance: 95 mm, Strength: 20551
Distance: 109 mm, Strength: 20836
Distance: 125 mm, Strength: 17752
Distance: 155 mm, Strength: 15763
Distance: 201 mm, Strength: 13696
Distance: 224 mm, Strength: 18329
Distance: 273 mm, Strength: 20668
Distance: 339 mm, Strength: 17926
Distance: 405 mm, Strength: 12878
Distance: 535 mm, Strength: 4080
Distance: 1268 mm, Strength: 4314
Distance: 1160 mm, Strength: 4474
Distance: 1129 mm, Strength: 4823
Distance: 1103 mm, Strength: 7231
```

- dis:距离 mm为单位
- strength:强度

5.SDM18的一些重要参数与默认值讲解

1. SDM18接口如图所示:

SDM18 对外物理接口端子为 WF08006，实现系统供电和数据通信功能。

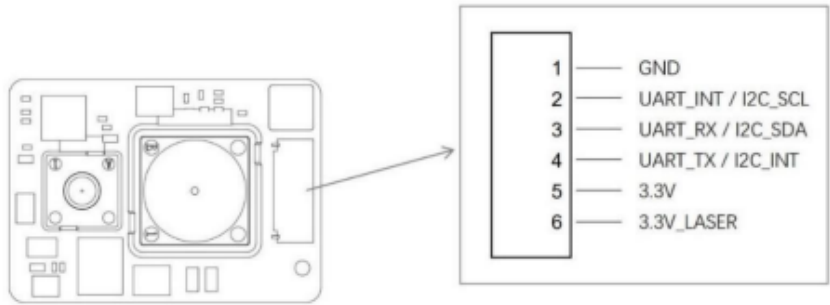


图 2 YDLIDAR SDM18 物理接口

2. 串口默认配置

SDM18默认的波特率为921600

接口	最小值	典型值	最大值	单位	备注
UART	9600	921600	921600	bps	信号电平 3.3V， 8 位数据位，1 位停止位， 无校验

3. 基于串口的重要协议命令(都是16进制的形式发送)

SDM18上电的默认模式是空闲模式，如果要进行测距要先开始测距的命令

- 开始测距: A5 03 20 01 00 00 00 02 6E
- 停止测距: A5 03 20 02 00 00 00 00 46 6E

接收测距的协议解析

包头	设备号	设备类型	命令类型	预留位	数据长度	数据段	校验码
1 Byte	1 Byte	1 Byte	1 Byte	1 Byte	2 Bytes	N Bytes	2 Bytes

- **包头**: SDM18 的报文包头标志为 0xA5;
- **设备号**: SDM18 的报文设备号标志为 0x03;
- **设备类型**: 根据下位机评估板的类型而定, 为 0x20;
- **命令类型**: 系统命令码, 见表 1;
- **预留位**: 预留状态位, 以留后续使用;
- **数据长度**: 表示的是应答数据的长度;
- **数据段**: 不同系统命令下的应答内容, 反馈不同的数据内容, 其数据格式也不同;
- **校验码**: 除去校验码以外, 所有数据的 CRC16 校验结果。

包头	设备号	设备类型	命令类型	预留位	数据长度	数据段	校验码
0xA5	0x03	0x20	0x01	0x00	0x00 0E		CRC16

设应答内容为: A5 03 20 01 00 00 0E FF FF FF FF FF FF F6 06 42 00 74 26 01 00 0B 44, 数据长度 = 0x00 0E, 即数据段字节数为 14; 则数据段为 FF FF FF FF FF FF F6 06 42 00 74 26 01 00, 其内容满足以下数据结构:

字节偏移	0	1	2	3	4	5	6	7	8	9	10	11	12	13
	距离值					强度值		预留位						

F6 06: 距离值为 0x06F6 = 1782mm

74 26: 强度值为 0x2674 = 9844

4. 校验码的计算

该校验码是才用了CRC-16-modelbus的计算方式, 把除了校验位以外的字节都进行计算

下图是这**开始测距**这条命令的校验位计算结果

输入内容

A5 03 20 01 00 00 00

内容格式

Hex

算法选择

CRC-16-MODBU

计算

清空

多项式公式

x16+x15+x2+1

宽度位数

16

多项式POLY(HEX)

8005

初始值INIT(HEX)

FFFF

结果异或值XOROUT(HEX)

0000

数据反转

输入反转(REFIN) ☒

输出反转(REFOUT) ☒

计算结果(HEX)

026E

计算结果(HEX)

00000010 01101110

其它协议不再讲解, 有兴趣的去看SDM18开发手册